



FACULTAD DE INGENIERÍA

Sistemas multiagente

Obligatorio 2: Juegos alternados.

2026

Profesor:

Marcos Adrian Begerez Oyhenard

Martín Iturbide Noria

Alumnos:

Natalia Campiglia - 349251

Marcos Mussio - 362839

Índice

1. Introducción	4
2. Marco Teórico	5
2.1. Juegos Alternados	5
2.2. Juegos de Suma Cero	5
2.3. Árboles de Juego	5
2.4. Estados de Información	6
2.5. Minimax	6
2.6. Monte Carlo Tree Search	6
2.7. Information Set Monte Carlo Tree Search	7
2.8. Counterfactual Regret Minimization	7
3. Ambientes	8
3.1. Tic-Tac-Toe	8
3.2. Nocco Nocco	9
3.3. Kuhn Poker	9
4. Implementación	10
4.1. Interfaz de Juegos Alternados	10
4.2. Agente Aleatorio	10
4.3. Minimax	10
4.4. MCTS	10
4.5. ISMCTS	11
4.6. CFR	11
5. Experimentos	12
5.1. Tic-Tac-Toe	12
5.2. Nocco Nocco	20

5.3. Kuhn Poker	25
6. Conclusión	26

1. Introducción

Los sistemas multiagente permiten modelar problemas donde varios agentes toman decisiones dentro de un ambiente compartido [5, 4]. En el Obligatorio 2 se trabaja con juegos alternados, es decir, ambientes en los que los agentes actúan por turnos y el estado del juego evoluciona luego de cada acción. Este tipo de interacción aparece en juegos de tablero, juegos de cartas y problemas secuenciales de decisión estratégica.

A diferencia de los juegos simultáneos, en los juegos alternados cada decisión puede depender del historial previo. Esto permite estudiar algoritmos de búsqueda, planificación y aprendizaje en juegos extensivos, tanto con información perfecta como con información imperfecta. En particular, este trabajo considera ambientes como Tic-Tac-Toe, Nocca Nocca y Kuhn Poker.

El objetivo del trabajo es implementar y analizar agentes capaces de jugar en estos ambientes. Para ello se desarrollaron agentes basados en Minimax, Monte Carlo Tree Search (MCTS), Information Set Monte Carlo Tree Search (ISMCTS), Counterfactual Regret Minimization (CFR) y un agente aleatorio de referencia.

El código del obligatorio se organiza alrededor de una abstracción común para juegos alternados, compatible con la interfaz de PettingZoo. Cada ambiente define acciones disponibles, observaciones, transiciones, recompensas y condiciones de finalización, mientras que los agentes consumen esa interfaz para seleccionar acciones.

El informe se organiza de la siguiente manera. Primero se presenta el marco teórico necesario para comprender juegos alternados, búsqueda adversarial y juegos con información imperfecta. Luego se describen los algoritmos implementados y los ambientes utilizados. Finalmente, se deja planteada la estructura de experimentos a completar con los resultados obtenidos.

2. Marco Teórico

2.1. Juegos Alternados

Un juego alternado es un juego secuencial en el que, en cada instante, un único agente tiene el turno de actuar. Luego de seleccionar una acción, el ambiente actualiza su estado, puede entregar recompensas y selecciona el siguiente agente. Esta dinámica se puede representar mediante un árbol de juego, donde los nodos corresponden a estados y las aristas a acciones.

En juegos de información perfecta, todos los agentes observan completamente el estado relevante del juego. Tic-Tac-Toe y Nocka Nocka pertenecen a esta familia. En juegos de información imperfecta, los agentes solo observan una parte del estado real. Kuhn Poker es un ejemplo clásico, porque cada jugador conoce su carta, pero no la carta del oponente.

2.2. Juegos de Suma Cero

Muchos juegos alternados competitivos se modelan como juegos de suma cero. En estos juegos, la ganancia de un agente implica una pérdida equivalente para el otro. Esta propiedad permite razonar en términos de maximización para el agente propio y minimización para el adversario.

2.3. Árboles de Juego

El árbol de juego representa todas las posibles continuaciones desde un estado inicial. Cada nodo contiene un estado, el agente que debe actuar y las acciones legales disponibles. Los nodos terminales representan estados finales y contienen las recompensas para cada agente.

En juegos pequeños, el árbol completo puede recorrerse de forma exhaustiva. Sin embargo, en juegos con mayor factor de ramificación o profundidad, recorrer todo el árbol resulta costoso, por lo que se usan heurísticas, límites de profundidad o métodos de muestreo.

2.4. Estados de Información

En juegos con información imperfecta, un agente no siempre puede distinguir entre todos los estados reales posibles. El conjunto de estados compatibles con lo que el agente observó se denomina estado de información. Los algoritmos para estos juegos deben decidir en función de ese conjunto, no únicamente de un estado real completamente observable.

Kuhn Poker introduce esta dificultad de forma mínima: cada jugador observa su carta privada y el historial público de apuestas, pero desconoce la carta privada del rival. Por eso resulta adecuado para evaluar algoritmos como ISMCTS y CFR.

2.5. Minimax

Minimax es un algoritmo clásico de búsqueda adversarial para juegos de dos jugadores, alternados, deterministas y de suma cero. La idea central es elegir la acción que maximiza la utilidad del agente propio, suponiendo que el oponente elegirá acciones que minimizan dicha utilidad.

La recurrencia puede expresarse como:

$$V(s) = \begin{cases} U(s), & \text{si } s \text{ es terminal} \\ \max_{a \in A(s)} V(T(s, a)), & \text{si actúa el agente propio} \\ \min_{a \in A(s)} V(T(s, a)), & \text{si actúa el oponente} \end{cases}$$

Cuando el árbol es demasiado grande, se puede limitar la profundidad de búsqueda y utilizar una función de evaluación heurística para estimar el valor de estados no terminales.

2.6. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) es un algoritmo de búsqueda que estima el valor de las acciones mediante simulaciones [1]. Cada iteración recorre cuatro etapas: selección, expansión, simu-

lación y retropropagación. La selección suele guiarse mediante UCB/UCT, balanceando explotación de acciones prometedoras y exploración de acciones poco visitadas.

MCTS resulta útil cuando el espacio de estados es grande y no se desea recorrer el árbol completo. En este obligatorio se usa como agente general para juegos alternados de información perfecta, realizando rollouts aleatorios desde estados expandidos.

2.7. Information Set Monte Carlo Tree Search

Information Set Monte Carlo Tree Search (ISMCTS) adapta MCTS a juegos con información imperfecta. En lugar de construir el árbol sobre estados reales completamente conocidos, el algoritmo opera sobre estados de información. En cada simulación se samplea una determinización compatible con lo observado por el agente y se ejecuta una búsqueda sobre ese posible estado.

Esta aproximación permite aplicar búsqueda Monte Carlo en juegos como Kuhn Poker, donde el agente debe razonar sobre cartas ocultas y decisiones del oponente sin conocer completamente el estado real.

2.8. Counterfactual Regret Minimization

Counterfactual Regret Minimization (CFR) es un algoritmo utilizado para aproximar estrategias en juegos extensivos con información imperfecta [3]. El algoritmo actualiza regrets por estado de información, comparando la utilidad esperada de las acciones disponibles contra la utilidad de la política actual.

La política se deriva de los regrets positivos acumulados mediante regret matching. Con suficientes iteraciones, en ciertos juegos de dos jugadores y suma cero, la estrategia promedio puede aproximarse a un equilibrio de Nash.

3. Ambientes

3.1. Tic-Tac-Toe

Tic-Tac-Toe es un juego alternado, determinista, de suma cero e información perfecta para dos jugadores. El tablero tiene dimensión 3×3 y los jugadores colocan marcas por turnos. El objetivo es completar una fila, columna o diagonal con tres marcas propias antes que el oponente.

En cada turno, el agente activo elige una acción entre las posiciones vacías del tablero. Como el tablero tiene nueve celdas, al inicio existen nueve acciones posibles. Luego de cada movimiento, el conjunto de acciones legales se reduce a las casillas todavía libres. La observación del agente contiene el estado completo del tablero, por lo que no existe información oculta ni incertidumbre sobre las jugadas anteriores.

Los valores terminales posibles para un agente son:

$$R_i = \begin{cases} 1, & \text{si el agente gana la partida} \\ -1, & \text{si el agente pierde la partida} \\ 0, & \text{si la partida termina en empate} \end{cases}$$

Como es un juego de suma cero, la recompensa del oponente es la opuesta cuando hay ganador. Si la partida termina en empate, ambos agentes reciben recompensa cero. En la implementación utilizada, estas recompensas se obtienen desde el ambiente al alcanzar un estado terminal.

Desde el punto de vista teórico, Tic-Tac-Toe es un juego resuelto. Si ambos jugadores actúan de manera óptima, el resultado esperado es empate. Por lo tanto, el valor del estado inicial bajo juego perfecto es:

$$V(s_0) = 0$$

Esto significa que ningún jugador puede forzar una victoria si el otro responde óptimamente. En términos de equilibrio, existe una estrategia óptima para cada jugador tal que ninguno puede mejorar unilateralmente su resultado esperado. Ese perfil de estrategias constituye un equilibrio de Nash en estrategias puras para el juego extensivo: frente a juego óptimo del rival, desviarse no permite obtener una recompensa mayor.

3.2. Nocca Nocca

Nocca Nocca es un juego de tablero alternado para dos jugadores. Cada jugador controla piezas sobre una grilla y puede moverlas de acuerdo con reglas de legalidad definidas por el tablero. El ambiente incorpora control de turnos, acciones legales, detección de finalización y recompensas según el ganador.

Este ambiente tiene un espacio de acciones más amplio que Tic-Tac-Toe y permite estudiar el comportamiento de algoritmos de búsqueda en un juego con mayor complejidad combinatoria.

3.3. Kuhn Poker

Kuhn Poker es un juego de cartas simplificado de dos jugadores y suma cero [2]. Se utiliza una baraja de tres cartas y cada jugador recibe una carta privada. Los jugadores alternan acciones de pasar o apostar, y el resultado depende del historial de apuestas y de las cartas privadas.

El valor del ambiente radica en que es un juego pequeño pero con información imperfecta. Cada agente conoce su propia carta y el historial público, pero no conoce la carta del oponente. Por este motivo, Kuhn Poker es especialmente adecuado para evaluar ISMCTS y CFR.

4. Implementación

4.1. Interfaz de Juegos Alternados

Los ambientes del obligatorio comparten una interfaz base llamada **AlternatingGame**. Esta abstracción define operaciones comunes como **reset**, **step**, **observe**, **available_actions**, **reward**, **terminated**, **truncated**, **game_over** y **clone**.

La operación **clone** es central para los algoritmos de búsqueda, ya que permite simular acciones futuras sin modificar el estado real del juego. Esta separación entre estado real y estado simulado es utilizada por Minimax, MCTS e ISMCTS.

4.2. Agente Aleatorio

El agente aleatorio selecciona uniformemente una acción entre las acciones legales disponibles. Se utiliza como línea base para comparar el desempeño de agentes más sofisticados.

4.3. Minimax

El agente Minimax recorre el árbol de juego desde el estado actual. Cuando el turno pertenece al agente propio, selecciona la acción que maximiza el valor esperado; cuando actúa el oponente, asume que este minimizará dicho valor. La implementación permite limitar la profundidad y utilizar una función de evaluación cuando no se alcanza un estado terminal.

4.4. MCTS

El agente MCTS construye un árbol de búsqueda a partir del estado actual. En cada simulación selecciona nodos mediante UCT, expande acciones no exploradas, ejecuta rollouts aleatorios y retropropaga las recompensas obtenidas. La acción final se elige a partir del valor medio estimado de los hijos de la raíz.

4.5. ISMCTS

El agente ISMCTS extiende la lógica de MCTS para trabajar con estados de información. Para cada simulación se obtiene un estado compatible con lo observado por el agente. Si el ambiente implementa una forma específica de samplear desde el conjunto de información, el agente la utiliza; en caso contrario, recurre a una copia del estado actual.

4.6. CFR

El agente CFR mantiene nodos asociados a observaciones o estados de información. En cada iteración recorre recursivamente el juego, calcula utilidades por acción y actualiza regrets contrafactuales. La política aprendida surge del promedio de las políticas generadas por regret matching.

5. Experimentos

5.1. Tic-Tac-Toe

En Tic-Tac-Toe se evaluaron MiniMax y Monte Carlo Tree Search (MCTS). Como es un juego pequeño y de información perfecta, permite comparar directamente una búsqueda por profundidad con una búsqueda basada en simulaciones.

El resultado esperado es empate, por eso, en los experimentos se toma el empate como referencia de equilibrio y analizamos cómo cambian los resultados al variar la profundidad de MiniMax, las simulaciones de MCTS y la cantidad de rollouts.

MiniMax vs MiniMax

El este primer bloque enfrentamos MiniMax contra MiniMax usando la misma profundidad para ambos agentes, variando `depth` de 1 a 9. Cada configuración se ejecutó con 3 partidas por lado (3: X y 3:O), es decir, 6 partidas por profundidad, esto lo hicimos para asegurarnos que no exista ventaja por quien comienza. Sabemos que en profundidades bajas, MiniMax debe cortar la búsqueda y usar la función heurística del ambiente, mientras que en profundidad 9 alcanza el árbol completo y representa juego óptimo.

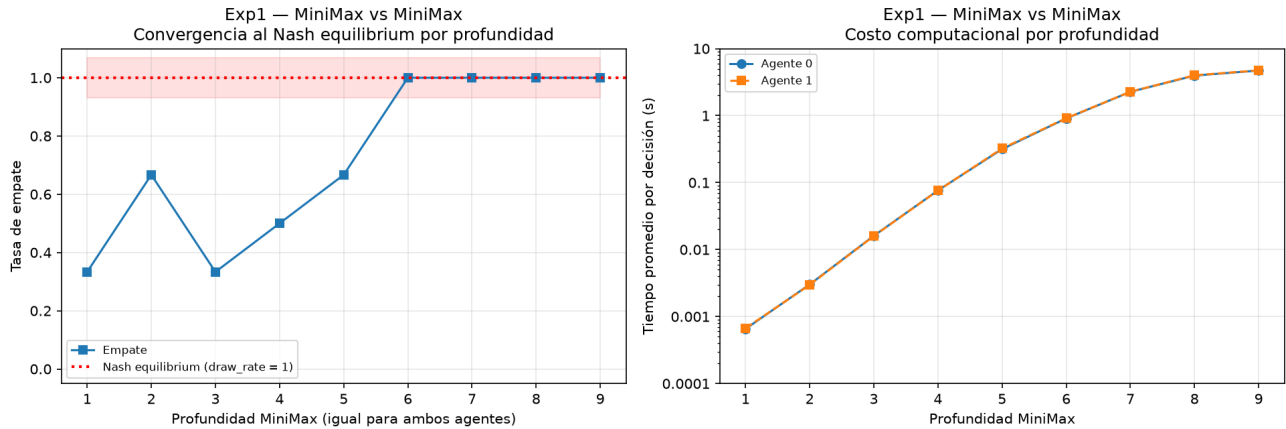


Figura 1: MiniMax vs MiniMax variando la profundidad.

Los resultados muestran que la tasa de empate aumenta al profundizar la búsqueda. Desde profundidad 6 en adelante todas las partidas terminaron en empate. Este es el comportamiento esperado, cuando ambos agentes tienen una búsqueda suficientemente profunda, ninguno logra forzar una victoria.

Por otro lado, se observa que el costo computacional crece rápidamente con la profundidad. El tiempo promedio de decisión pasa de menos de 1ms en profundidad 1 a aproximadamente 4.7s en profundidad 9. Por este motivo, para Tic-Tac-Toe no siempre resulta necesario usar la profundidad completa, como vemos con profundidades intermedias, como profundidad 6 en esta corrida, ya alcanzan el equilibrio y reducen de forma importante el gasto computacional.

MCTS vs MCTS: simulaciones

El segundo bloque enfrenta MCTS contra MCTS usando el mismo presupuesto para ambos jugadores, con `rollouts=10` fijo y variando las simulaciones en [10, 25, 50, 100, 200, 300, 500]. En cada punto se jugaron 50 partidas por lado, para un total de 100 partidas por configuración.

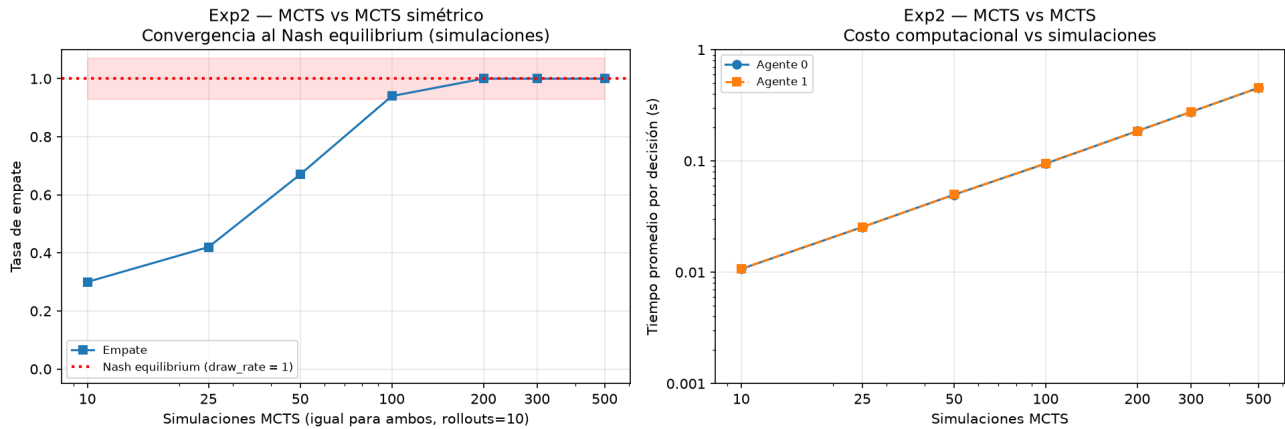


Figura 2: MCTS vs MCTS variando la cantidad de simulaciones.

La tasa de empate aumenta claramente con el número de simulaciones. Con 10 simulaciones fue 30 %, con 50 simulaciones subió a 67 %, con 100 simulaciones llegó a 94 %, y desde 200 simulaciones

todas las partidas terminaron en empate. Esto muestra una convergencia empírica hacia el equilibrio de Nash cuando ambos agentes MCTS disponen de suficiente simulaciones.

El costo computacional también aumenta de forma clara con el número de simulaciones, aproximadamente 11ms por decisión con 10 simulaciones y 1s con 500 simulaciones. Esto confirma que el presupuesto de simulaciones afecta directamente el tiempo de decisión de MCTS.

MCTS vs MCTS: rollouts

El tercer bloque mantiene `simulations=150` y varía la cantidad de rollouts entre 1 y 20, usando los valores 1, 2, 5, 8, 10, 12, 15, 18 y 20. Igual que antes, ambos agentes usan el mismo presupuesto y el agente 0 juega 80 partidas como X y 80 como O.

Los resultados muestran que aumentar rollouts mejora la estabilidad de MCTS hasta acercarse mucho al equilibrio. Con 1 rollout la tasa de empate fue 71.25 %, con 5 rollouts fue 97.5 %, y con 15 y 18 rollouts alcanzó 100 %. A partir de ese punto, agregar rollouts no aporta una mejora visible en resultado, pero sí aumenta el costo, de aproximadamente 32ms por decisión con 1 rollout a 259ms con 20 rollouts.

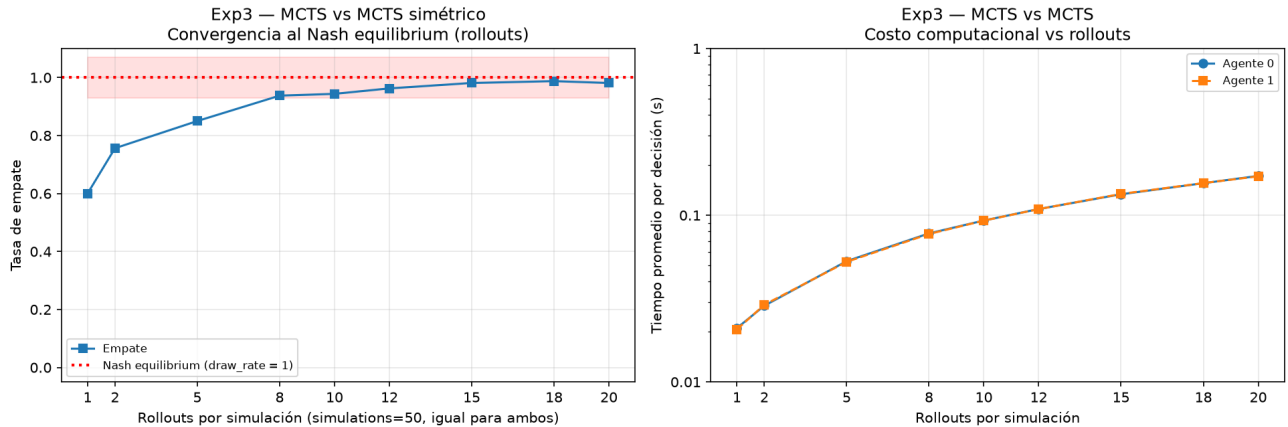


Figura 3: MCTS vs MCTS variando la cantidad de rollouts.

MCTS vs MiniMax

En este bloque comparamos MCTS contra MiniMax. El agente 0 es MCTS, con simulaciones en $[50, 80, 100, 200]$ y `rollouts=12`; el agente 1 es MiniMax, con profundidades de 1 a 6. Para cada combinación se jugaron 20 partidas por lado.

Simulaciones MCTS	Profundidad MiniMax	Gana MCTS	Empates	Gana MiniMax	Reward prom. MCTS
50	1	31	9	0	0.775
50	2	0	40	0	0.000
50	3	9	27	4	0.125
50	4	9	26	5	0.100
50	5	10	18	12	-0.050
50	6	0	35	5	-0.125
80	1	31	9	0	0.775
80	2	0	38	2	-0.050
80	3	9	31	0	0.225
80	4	10	30	0	0.250
80	5	13	24	3	0.250
80	6	0	38	2	-0.050
100	1	34	6	0	0.850
100	2	0	40	0	0.000
100	3	6	33	1	0.125
100	4	8	32	0	0.200
100	5	10	29	1	0.225
100	6	0	39	1	-0.025
200	1	32	8	0	0.800
200	2	0	40	0	0.000
200	3	12	28	0	0.300
200	4	6	33	1	0.125
200	5	12	28	0	0.300
200	6	0	40	0	0.000

Tabla 1: Resultados de MCTS vs MiniMax. Cada fila resume 40 partidas desde la perspectiva de MCTS.

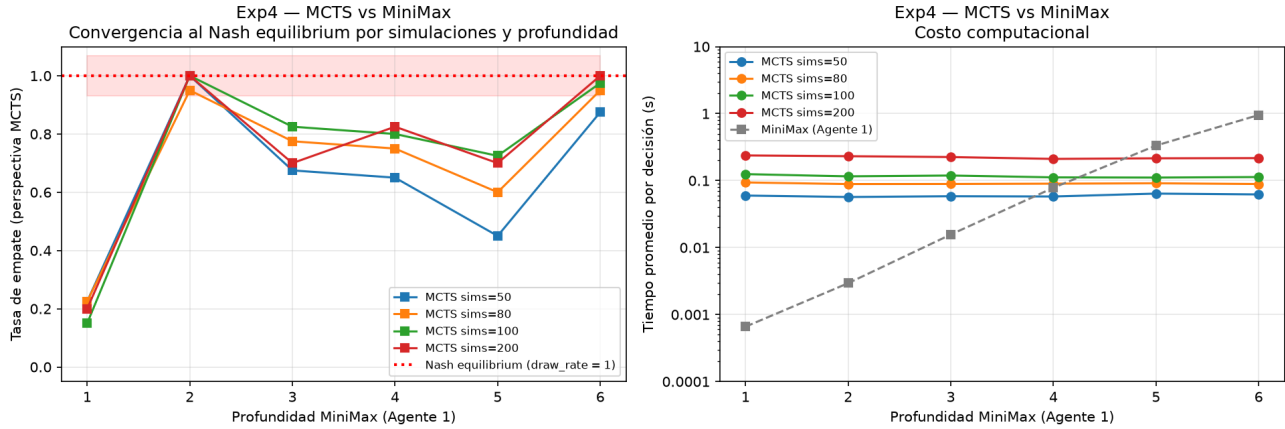


Figura 4: MCTS vs MiniMax variando simulaciones y profundidad.

Contra MiniMax de profundidad 1, MCTS obtiene una ventaja clara, gana entre 77.5% y 85% de las partidas. Contra profundidad 2, el resultado es casi siempre empate, con 50, 100 y 200 simulaciones todas las partidas empataron, y con 80 simulaciones hubo 38 empates y 2 derrotas. Contra profundidades 3 a 5, MCTS obtiene resultados positivos en varias configuraciones, aunque con variación por la naturaleza estocástica de los rollouts, estos completan partidas simuladas de forma aleatoria y, por lo tanto, el valor estimado de una acción puede cambiar levemente entre corridas, aun partiendo de una posición similar. Contra profundidad 6, MCTS no consigue victorias, y logra llegar al equilibrio al aumentar las simulaciones, con 50 simulaciones hubo 35 empates y 5 derrotas, mientras que con 200 simulaciones las 40 partidas terminaron empatadas.

El gráfico de tiempos confirma los costo de ambos algoritmos. Para MiniMax, pasar de profundidad 1 a profundidad 6 implica pasar de menos de 1 ms a cerca de 1s por decisión, incluso en un juego tan pequeño como Tic-Tac-Toe, sin embargo los costos computacionales en MCTS no cambian tanto, estando todos al rededor de 0.2s.

MCTS vs MiniMax óptimo

Como contraste final se enfrentó MCTS con 200 simulaciones y 10 rollouts contra MiniMax con profundidad 9. Se jugaron 3 partidas con MCTS como X y 3 como O, debido al costo de la búsqueda completa sin poda alfa-beta.

Las 6 partidas terminaron en empate: 0 % victorias de MCTS, 100 % empates y 0 % derrotas. Este resultado coincide con el valor minimax de Tic-Tac-Toe. Contra un oponente óptimo no se puede forzar una victoria, pero con presupuesto suficiente MCTS logró sostener el empate.

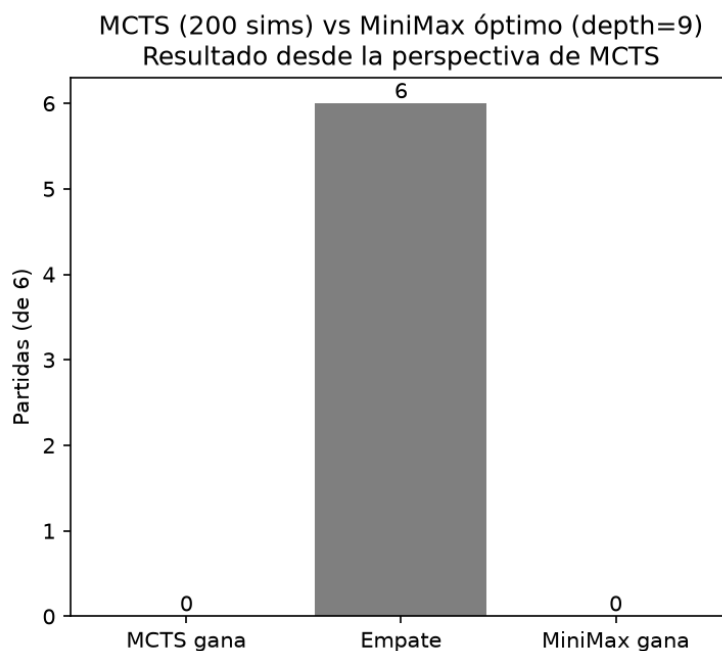


Figura 5: MCTS contra MiniMax óptimo con profundidad completa.

Benchmark de tiempos

Además de medir desempeño en partidas, se comparó el tiempo promedio de decisión de los agentes. Para que el benchmark fuera comparable, todos los agentes jugaron la misma cantidad de partidas como O contra Random como X.

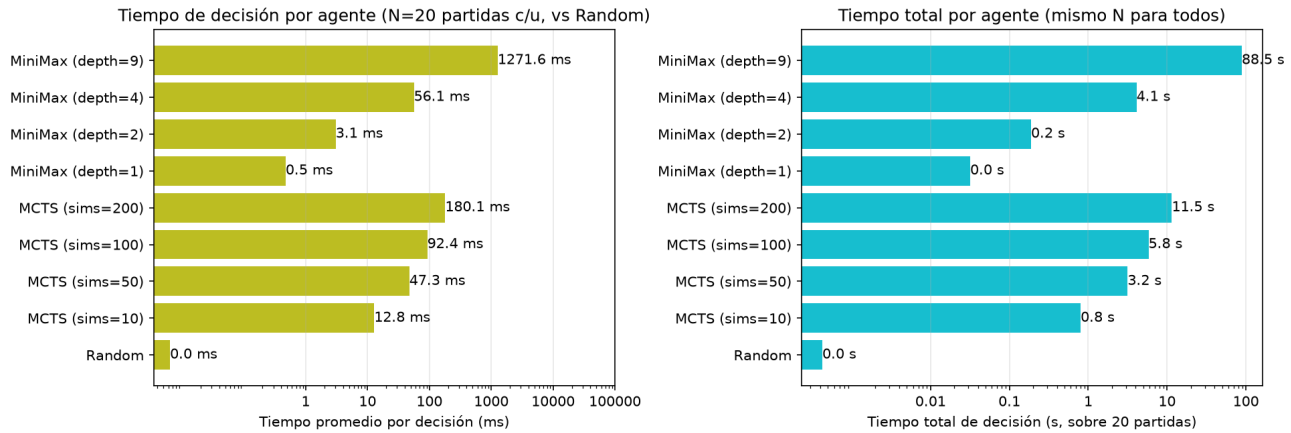


Figura 6: Comparación de tiempos de decisión entre agentes.

El tiempo promedio por decisión de MiniMax pasó de aproximadamente 0.00047 s con profundidad 1 a 1.272 s con profundidad 9, un aumento de varios órdenes de magnitud. MCTS, en cambio, escaló de forma más suave, de 0.0128 s con 10 simulaciones a 0.1801 s con 200 simulaciones. En tiempo total sobre 20 partidas, MiniMax con profundidad 9 insumió alrededor de 88.5 s, mientras que MCTS con 200 simulaciones tomó alrededor de 11.5 s.

Conclusiones de Tic-Tac-Toe

Los experimentos muestran que Tic-Tac-Toe funciona como un ambiente de validación adecuado para MiniMax y MCTS. MiniMax permite representar desde un agente heurístico con profundidad limitada hasta un jugador óptimo con profundidad completa. En self-play, MiniMax converge claramente al empate al aumentar la profundidad, y MCTS también converge al empate cuando se aumenta el presupuesto de simulaciones o se usan suficientes rollouts.

La comparación con MiniMax ilustra el compromiso central entre búsqueda exhaustiva y búsqueda por muestreo. En Tic-Tac-Toe, MiniMax con profundidad completa es posible y permite recuperar exactamente el equilibrio de Nash del juego. Sin embargo, su costo crece rápidamente con la profundidad. MCTS no garantiza exactitud en cada decisión y sus resultados dependen de

la varianza de las simulaciones: como los rollouts completan partidas de forma aleatoria, distintas simulaciones pueden producir estimaciones algo diferentes para una misma posición. Aun así, MCTS permite regular el costo computacional mediante **simulations** y **rollouts**. Esta diferencia es la razón por la que MCTS se vuelve especialmente atractivo en juegos con árboles más grandes, donde una búsqueda exhaustiva deja de ser viable.

5.2. Nocca Nocca

En Nocca Nocca se evaluaron principalmente MiniMax y Monte Carlo Tree Search (MCTS). A diferencia de Tic-Tac-Toe, este juego tiene un tablero más grande, más acciones legales por estado y partidas potencialmente más largas. Por eso no se usa una solución exacta del juego como referencia, sino que se comparan resultados empíricos y tiempos de decisión.

Los experimentos se realizaron desde la perspectiva del agente evaluado. En las comparaciones simétricas, el agente juega la misma cantidad de partidas como jugador inicial y como segundo jugador, y luego se combinan los resultados. Esta evaluación ayuda a reducir el sesgo asociado al orden de juego. Como no se dispone de un valor de Nash calculado para Nocca Nocca, la recompensa promedio 0 se usa únicamente como referencia neutral: valores positivos indican ventaja del agente evaluado y valores negativos indican ventaja del rival.

MiniMax vs MiniMax

Luego se evaluó MiniMax contra sí mismo usando profundidades 1, 2 y 3. En cada configuración se jugaron 5 partidas por cada rol, para un total de 10 partidas. A diferencia de Tic-Tac-Toe, MiniMax no puede buscar hasta el final de la partida. Además, la función de evaluación utilizada devuelve recompensa terminal cuando el juego termina y 0 en estados no terminales. Por eso, con profundidades acotadas, MiniMax solo distingue con claridad posiciones donde aparece una victoria o derrota dentro del horizonte de búsqueda.

Profundidad	Victorias A0	Empates	Victorias A1	Recompensa A0	Tiempo A0 (ms)
1	4	1	5	-0.10	2.39
2	3	3	4	-0.10	69.68
3	4	0	6	-0.20	1826.14

Tabla 2: Resultados de MiniMax vs MiniMax en Nocca Nocca.

Los resultados no muestran una convergencia clara al aumentar la profundidad: la recompensa promedio del agente 0 se mantiene cerca de 0, pero levemente negativa en las tres configuraciones. Esto se explica por la combinación de horizonte limitado, alta ramificación y una evaluación poco informativa en estados intermedios. En cambio, el costo computacional sí crece de forma marcada.

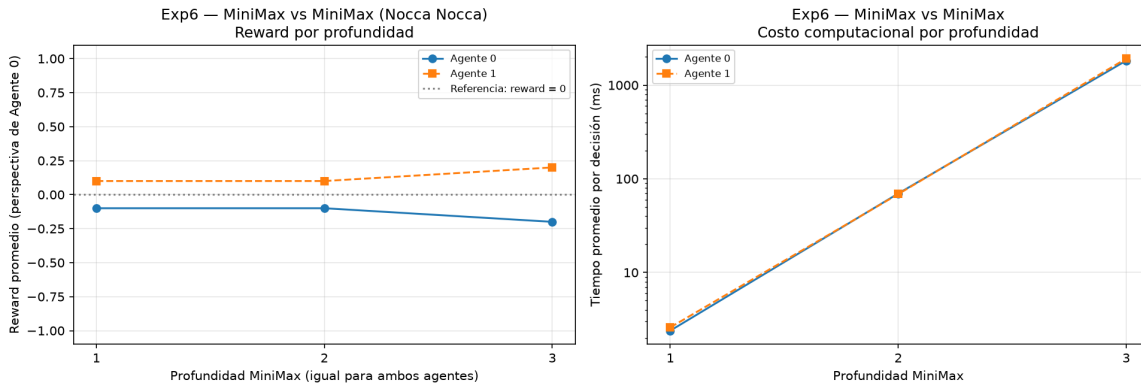


Figura 7: MiniMax vs MiniMax con evaluación simétrica.

MCTS vs MCTS

Para analizar MCTS en self-play se realizaron dos barridos. En el primero se varió la cantidad de simulaciones, manteniendo 5 rollouts por acción. Se probaron 25, 50, 100, 200 y 500 simulaciones, con 10 partidas por rol en cada configuración.

Simulaciones	Victorias A0	Empates	Victorias A1	Recompensa A0	Tiempo A0 (ms)
25	8	0	12	-0.20	1451.10
50	12	0	8	0.20	3088.79
100	10	0	10	0.00	6251.94
200	11	0	9	0.10	12295.90
500	10	0	10	0.00	30683.24

Tabla 3: MCTS vs MCTS variando simulaciones.

La recompensa promedio oscila alrededor de 0, lo que indica que ningún rol domina de manera estable en esta corrida. El resultado más relevante es el crecimiento del tiempo de decisión: pasar de 25 a 500 simulaciones aumenta el costo desde aproximadamente 1.45 s hasta más de 30 s por decisión. En este juego, por lo tanto, aumentar simulaciones mejora la exploración, pero rápidamente vuelve costosa cada jugada.

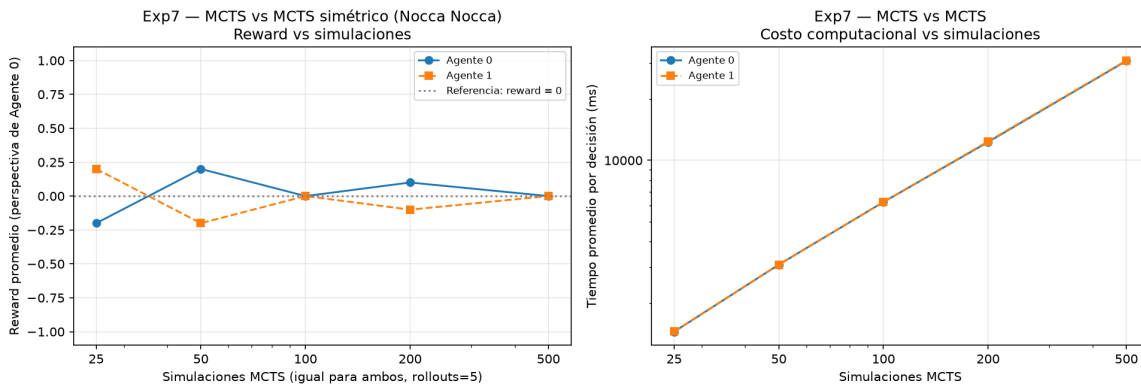


Figura 8: MCTS vs MCTS variando la cantidad de simulaciones.

En el segundo barrido se fijaron 100 simulaciones y se varió la cantidad de rollouts por acción. Se probaron 1, 5, 10 y 20 rollouts. En principio, usar más rollouts puede reducir la varianza de la estimación de cada acción, porque cada nodo se evalúa con más partidas simuladas. Sin embargo, en

la corrida obtenida los resultados no fueron monótonos: con 5 rollouts el agente 0 obtuvo su mejor recompensa promedio, mientras que con 10 rollouts volvió a quedar levemente por debajo de 0. Esto refleja que, con pocas partidas por configuración, la aleatoriedad de los rollouts y las decisiones tempranas todavía puede afectar el resultado agregado.

Rollouts	Victorias A0	Empates	Victorias A1	Recompensa A0	Tiempo A0 (ms)
1	8	0	12	-0.20	1133.80
5	15	0	5	0.50	6040.34
10	9	0	11	-0.10	12816.50
20	12	0	8	0.20	25610.21

Tabla 4: MCTS vs MCTS variando rollouts.

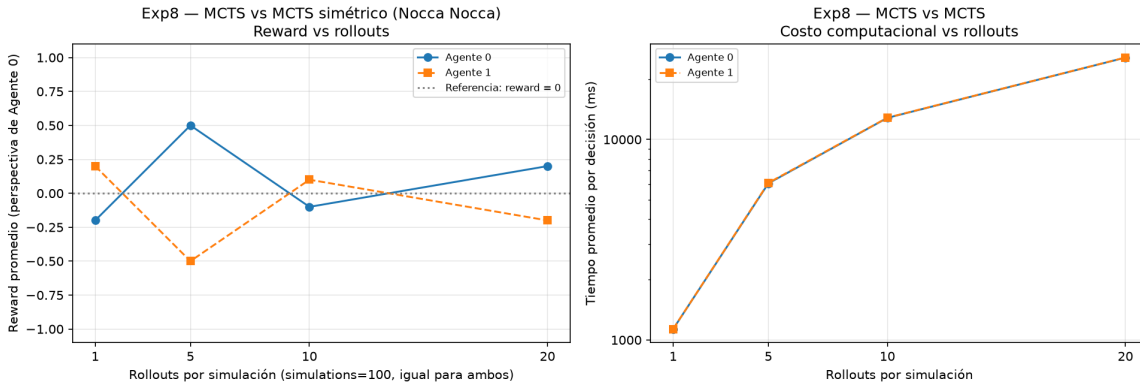


Figura 9: MCTS vs MCTS variando la cantidad de rollouts.

MCTS vs MiniMax

También se comparó MCTS contra MiniMax. En esta evaluación, MiniMax representa una búsqueda determinista acotada en profundidad, mientras que MCTS representa una búsqueda aproximada basada en simulaciones. Contra MiniMax de baja profundidad, MCTS obtiene ventaja con facilidad. Contra profundidad 3, el resultado se vuelve más competitivo: con pocas simulaciones

MCTS puede quedar por debajo, pero al aumentar el presupuesto logra acercarse e incluso superar a MiniMax en algunas configuraciones.

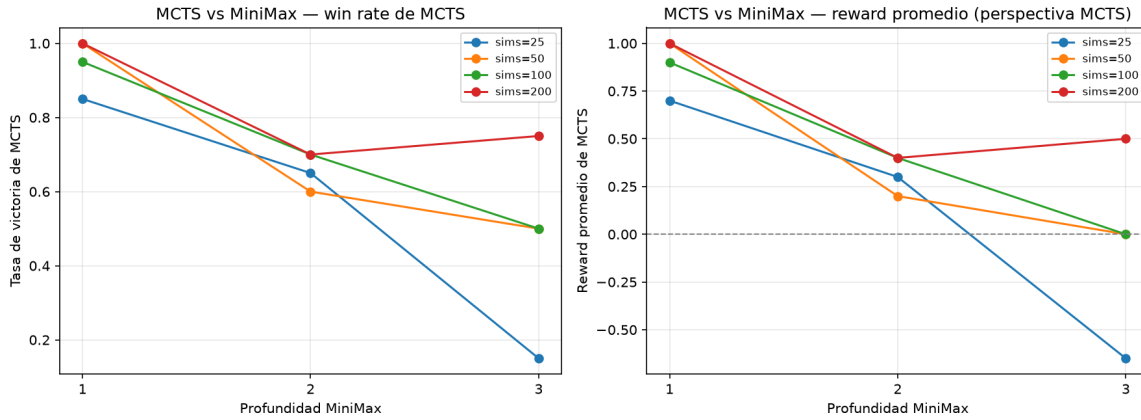


Figura 10: MCTS contra MiniMax en Noca Noca.

Conclusiones de Noca Noca

Noca Noca muestra mejor que Tic-Tac-Toe el compromiso entre calidad de decisión y costo computacional. MiniMax mejora al aumentar profundidad, pero el crecimiento del árbol hace que profundidades mayores sean caras y que una evaluación terminal simple no alcance para ordenar bien estados intermedios. MCTS, en cambio, permite controlar el presupuesto mediante simulaciones y rollouts. Su desventaja es que el costo crece con partidas simuladas largas y que los resultados pueden variar entre corridas por la aleatoriedad de los rollouts.

En conjunto, los experimentos sugieren que para Noca Noca MCTS es más flexible que MiniMax con profundidad fija, especialmente cuando no se cuenta con una heurística fuerte para evaluar estados no terminales. Aun así, el presupuesto debe elegirse con cuidado: más simulaciones o más rollouts no garantizan una mejora monótona en pocas partidas, pero sí aumentan de forma clara el tiempo de decisión.

5.3. Kuhn Poker

Random vs Random

CFR vs Random

ISMCTS vs Random

CFR vs ISMCTS

Política aprendida por CFR

Análisis de información imperfecta

6. Conclusión

Este obligatorio permite estudiar problemas de decisión secuencial multiagente mediante juegos alternados. A partir de una interfaz común para ambientes, se implementaron agentes representativos de distintas familias: búsqueda adversarial exhaustiva, búsqueda Monte Carlo, búsqueda sobre estados de información y minimización de regret.

Tic-Tac-Toe ofrece un punto de partida controlado para verificar el funcionamiento de los algoritmos en información perfecta. Nocca Nocca introduce un espacio de acciones más complejo, donde la eficiencia de la búsqueda se vuelve más relevante. Kuhn Poker agrega información imperfecta y permite observar la necesidad de razonar sobre estados compatibles con la observación del agente.

En conjunto, los ambientes y agentes implementados muestran cómo la estructura del juego condiciona la elección del algoritmo. Minimax resulta natural para juegos pequeños de información perfecta, MCTS permite escalar mediante simulaciones, ISMCTS adapta la búsqueda a información imperfecta y CFR permite aprender políticas promedio en juegos extensivos.

Referencias

- [1] Levente Kocsis and Csaba Szepesvari. Bandit based monte-carlo planning. *European Conference on Machine Learning*, 2006.
- [2] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*. Princeton University Press, 1950.
- [3] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte carlo sampling for regret minimization in extensive games. *Advances in Neural Information Processing Systems*, 2009.
- [4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 2021.

- [5] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.